

The vertex separation number of a graph equals its path-width

Nancy G. Kinnersley *

Department of Computer Science, University of Kansas, Lawrence, KS 66045, USA

Communicated by D. Dolev

Received 18 July 1990

Revised 2 January 1992

Abstract

Kinnersley, N.G., The vertex separation number of a graph equals its path-width, Information Processing Letters 42 (1992) 345–350.

We show that the problems of finding a graph's vertex separation number and path-width are the same. The equivalence of these problems with the gate matrix layout and the node search number problems then follows immediately from the results of Fellows and Langston and Kirousis and Papadimitriou, respectively. The fixed parameter variants of problems are known to possess $O(n^2)$ decision algorithms based on finite but unknown obstruction sets. We show how all tree obstructions in these sets may be constructively obtained.

Keywords: Combinatorial problems, computational complexity, theory of computation

1. Introduction

GATE MATRIX LAYOUT [2], VERTEX SEPARATION [11], SEARCH NUMBER [12], and its variant NODE SEARCH NUMBER [9] are NP-complete combinatorial problems that arise in VLSI design. Path-width is an important structural metric in the Robertson–Seymour theory. Kirousis and Papadimitriou [9] proved that the node search number of a graph is one more than its vertex separation number, and Fellows

and Langston [6] showed that any graph with gate matrix layout cost k has path-width $k - 1$. In this paper we show that the vertex separation number of a graph equals its path-width, thereby proving the equivalence of the four problems. Moreover, using a simple transformation [3] and an algorithm for any of the four problems, the search number of a graph can be found.

The Robertson–Seymour theorems (see, for example, [13,14]) have been used by Fellows and Langston to show that the fixed parameter variants of each of the above problems have low-degree polynomial-time decision complexity [4–6]. Each decision algorithm depends on the existence of a finite collection of graphs known as an obstruction set. While there is no general method of identifying all graphs in an obstruction set, we show how, for these problems, the trees in each set can be obtained.

Correspondence to: N.G. Kinnersley, Department of Computer Science, University of Kansas, Lawrence, KS 66045-2192, USA.

* This author's research has been supported in part by the University of Kansas under general research allocation 3800-20-0038 and by the National Science Foundation under grant CCR-9008725.

2. Background

The combinatorial problem GATE MATRIX LAYOUT (GML) [2] is the archetypical example used by Fellows and Langston [4–6] to illustrate the application of Robertson and Seymour poset techniques. Formally, given an $n \times m$ Boolean matrix M and a positive integer k , we ask whether the columns of M can be permuted so that, if in each row we change to * every 0 lying between the row's leftmost and rightmost 1, then no column contains more than k 1s and *s. As described in [4] an instance of the problem can be mapped to an equivalent instance with exactly two ones per column and then modeled as a graph in which the rows correspond to vertices and the columns to edges. Thus, a permutation of the columns of an incidence matrix corresponds to a permutation of the edges of a graph. The cost of a permutation is the maximum number of 1s and *s in any column of the corresponding matrix. The (optimal) gate matrix layout cost of G , denoted by $gml(G)$, is the minimum cost of any of its permutations. We use the notation k -GML for the fixed parameter variants of this problem.

In a search game the objective is to capture a fugitive who moves freely about the edges of a graph G . Initially all edges are considered *contaminated*. A search strategy is a sequence of steps that will clear all edges of a graph. To clear an edge a searcher is placed at one end and a second searcher moves along the edge from the guarded endpoint to the other endpoint. If all other edges incident on the guarded endpoint are already clear, then the guard may be moved along the edge to the other endpoint. The (*edge*) *search number* [12] of G , denoted by $s(G)$, is the minimum number of searchers for which a search strategy exists. In the *node-searching* version of the problem [9], searchers are used only to guard nodes, and an edge becomes clear when both of its endpoints are concurrently guarded by searchers. The *node search number* is denoted by $ns(G)$.

A (*linear*) *layout* of a graph $G = \langle V, E \rangle$ is a one to one mapping $L: V \rightarrow \{1, 2, \dots, |V|\}$. Thus, L is a permutation of the vertices of G . For any

layout L , define $V_L(i) = \{u \in V \mid L(u) \leq i \text{ and there is some } v \in V \text{ such that } uv \in E \text{ and } L(v) > i\}$. $V_L(i)$ is the number of vertices of G mapped to integers less than or equal to i that are adjacent to vertices mapped to integers greater than i . The *vertex separation number* of G with respect to L , $vs_L(G)$, is the maximum number of vertices in any $V_L(i)$. The vertex separation number of G is the minimum, over all possible layouts L of G , of $vs_L(G)$. Formally,

$$vs_L(G) = \max_{1 \leq i < |V|} \{|V_L(i)|\}$$

and

$$vs(G) = \min\{vs_L(G) \mid L \text{ is a linear layout of } G\}.$$

Given both a graph g and a positive integer k , the NP-complete VERTEX SEPARATION problem [11] asks if $vs(G)$ is less than or equal to k .

Given a graph $G = \langle V, E \rangle$, a sequence $X_1, \dots, X_r$ of subsets of V is a *path-decomposition* of G if the following conditions are satisfied:

- (a) $\cup X_i = V$,
- (b) for every edge e of E , some X_i contains both endpoints of e , and
- (c) for $1 \leq i \leq j \leq k \leq r$, $X_i \cap X_k \subseteq X_j$.

The *path-width* of G , denoted by $pw(G)$, is the minimum value $h \geq 0$ such that G has a path-decomposition $X_1, X_2, \dots, X_r$ with $|X_i| \leq h + 1$ for $i = 1, \dots, r$.

3. The main result

Theorem 3.1 shows that a graph with path-width k also has vertex separation k . Since multiple edges and loops do not increase a graph's path-width or vertex separation number, we consider only simple, connected graphs.

Theorem 3.1. *For any graph G , $vs(G) = pw(G)$.*

Proof. Let $G = \langle V, E \rangle$ be a graph on n vertices. Suppose $vs(G) = k$ and let L be a linear layout

of G such that $vs_L(G) = k$. Define sets $X_1, \dots, X_n$ as follows:

$$X_1 = \{L^{-1}(1)\},$$

$$X_2 = V_L(1) \cup \{L^{-1}(2)\},$$

...

$$X_n = V_L(n-1) \cup \{L^{-1}(n)\}.$$

Let $e = uv$ be any edge in E and assume $L(u) = i$ and $L(v) = j$ with $i < j$. Then $u \in V_L(j-1) \subseteq X_j$ and $v \in X_j$ so both endpoints of e are in X_j .

Now, suppose $1 \leq i < j < m \leq n$ and $X_i \cap X_m \neq \emptyset$. Suppose $a \in X_i \cap X_m$, and $L(a) = s$. Then $a \in V_L(i-1) \cup \{u\}$ and $s \leq i$. Since $a \in X_m$, then $a \in V_L(m-1)$. Thus, there is a $t \geq m$ such that $L^{-1}(t) = b$ and $ab \in E$. Since $s < j < t$, $a \in V_L(j-1) \subseteq X_j$. Therefore, $X_1, \dots, X_n$ is a path-decomposition of G .

Since for $i = 1, \dots, n-1$, $|V_L(i)| \leq k$, it follows that $|X_i| \leq k+1$ for every $i = 1, \dots, n$ and G has path-width at most k . Thus, for any graph G , $pw(G) \leq vs(G)$.

Now, suppose G has path-width k and let $X_1, \dots, X_r$ be a path-decomposition of G with $|X_i| \leq k+1$ for $i = 1, \dots, r$. We now construct a linear layout L of G . Suppose $|X_1| = f$. Let L be a one to one mapping from X_1 to $\{1, \dots, f\}$. Extend the mapping L to all of V as follows:

```

 $V' \leftarrow X_1$ 
For  $i \leftarrow 2$  to  $r$  do
   $Y \leftarrow X_i \setminus V'$ 
  While  $Y \neq \emptyset$  do
     $L(y) \leftarrow |V'| + 1$  for arbitrary  $y \in Y$ 
     $V' \leftarrow V' \cup \{y\}$ 
     $Y \leftarrow Y \setminus \{y\}$ 
  end while
end for

```

Since G is connected, every vertex of G is in some X_i and hence L is a linear layout of G .

To determine $vs_L(G)$ we look at $V_L(i)$ for $1 \leq i < n$. Let X_m be the first set in the decomposition that contains $L^{-1}(i)$, and let $a = L^{-1}(s)$ be any element of $V_L(i)$. By the definition of $V_L(i)$, we know that $s \leq i$ and there is some $t > i$ such

that $L^{-1}(t) = b$ and ab is an edge of G . Since $s \leq i$, there is an $h \leq m$ such that $a \in X_h$. Since $ab \in E$ and $t > i$ there is some $j \geq m$ such that X_j contains both a and b . Therefore, $a \in X_h \cap X_j$. Since $h \leq m \leq j$, the intersection property of the path-decomposition guarantees that $a \in X_m$. Thus, $V_L(i) \subseteq X_m$.

Since we may assume that no set in a path-decomposition is a subset of another, there is some element c that is in X_m but not in X_{m+1} . This means cd , the last edge incident on c , has both endpoints in X_m . Thus, if $V_L(i) \subseteq X_m$ then $c \notin V_L(i)$, or $d \notin V_L(i)$. Therefore, $|V_L(i)| \leq |X_m| - 1$.

Therefore, for every $i = 1, \dots, n-1$, there is some m , $1 \leq m \leq r$ such that $V_L(i) \subseteq X_m$ and $|V_L(i)| \leq |X_m| - 1 \leq k+1-1 = k$. Thus, $vs(G) \leq k$, and hence, for any graph, its vertex separation number and path-width are the same. $\square$

Corollary 3.2. *Let G be a graph. If $vs(G) = k$, then $gml(G) = ns(G) = k+1$.*

Proof. The proof follows immediately from Theorem 3.1 above, Theorem 7 of [6], and Theorem 4.1 of [9]. $\square$

We may also use the result that recontamination does not help in searching a graph [10] to describe how a path-decomposition yields a search strategy and vice versa. In [8] it was shown that a node-search strategy satisfying the following conditions is recontamination-free: (1) a node v gets a searcher only once, and (2) the searcher is removed from v after all vertices in the neighborhood of v have held searchers. Then, defining X_i to be the set of nodes in G that hold a searcher at stage i in a search strategy produces a path-decomposition of G . The first condition ensures that the intersection requirement is met, while the second guarantees that each edge has its endpoints in some X_i . If a path-decomposition for G is given, place a searcher at each node in X_1 . Then, at step i , $1 \leq i < r$, remove the searcher from each node in $X_i \setminus X_{i+1}$ and place a searcher on each node in $X_{i+1} \setminus X_i$. This strategy will clear the edges of the graph using no more than $k+1$ searchers, where k is the path-width of G . The

(edge) searcher number of a graph G may be found by using the result in [3] that $s(G) = vs(S_2(G))$, where $S_2(G)$ is obtained by inserting two new degree-two vertices into each edge of G .

4. Obstruction set results

Nonconstructive methods based on advances in well-partial-order theory have been used to establish polynomial-time decision complexity for the fixed parameter variants of the problems discussed above [4–6]. For each value of k , the decision algorithm is based on the existence of a finite collection of graphs termed an obstruction set. Each obstruction has cost greater than k , and is minimal in the minor order.¹ The theory itself guarantees only the finiteness of the set, and gives no indication of how many members the set contains or how to find them. Nonetheless, the obstruction sets for 2-GML and 3-GML have been identified and contain two [1,4] and 110 elements [7], respectively.

While there are no algorithms that enable us to directly identify all elements of the k -GML obstruction sets, we can precisely describe how to *constructively* obtain the tree obstructions in each set. We begin with the following result of Ellis, Sudborough and Turner on the vertex separation of trees:

Theorem 4.1 [3]. *Let T be a tree and $k \geq 1$. $vs(T) \leq k$ if and only if for all vertices x in T at most two of the subtrees induced by x have vertex separation k and all other subtrees have vertex separation $\leq k - 1$.*

Corollary 4.2 follows immediately from Corollary 3.2 and Theorem 4.1.

¹ A graph H is less than or equal to a graph G in the minor order, written $H \leq_m G$, if and only if a graph isomorphic to H can be obtained from G by a series of these two operations: taking a subgraph and contracting an arbitrary edge. Since an obstruction is minimal, each of its proper minors has cost at most k .

Corollary 4.2. *Let T be a tree and $k \geq 2$. $gml(T) > k$ if and only if there is a vertex $v \in T$ that induces three or more subtrees each with cost at least k .*

It was shown in [4] that if a new vertex v is made adjacent to one vertex in each of three graphs with gate matrix layout cost k , then the resulting graph has cost at least $k + 1$. As we show next, this construction preserves minimality, thereby guaranteeing that if the original graphs are obstructions for k -GML, then the new graph is an obstruction for $(k + 1)$ -GML.

Theorem 4.3. *Let G_1, G_2 and G_3 denote disjoint (but not necessarily distinct) obstructions for k -GML, let v_i denote an arbitrary vertex of G_i for $1 \leq i \leq 3$, and let v denote an isolated vertex not in $G_1 \cup G_2 \cup G_3$. The graph $G = G_1 \cup G_2 \cup G_3 \cup \{v\} \cup \{vv_i : 1 \leq i \leq 3\}$ is an obstruction for $(k + 1)$ -GML.*

Proof. Let G_i, v_i, v and G be as defined in the statement of the lemma. It follows from Lemma 4.3 of [4] that G has cost $k + 2$. If $u_i v_i$ is any edge of G_i incident on v_i , then the subgraph $G_i \setminus \{u_i v_i\}$ possesses a cost k permutation. Then, for each such permutation, placing $u_i v_i$ adjacent to any column with a 1 in row u_i or row v_i , produces, for G_i , a cost $k + 1$ permutation M_i in which every column with cost $k + 1$ has a 1 or an * in row v_i .

We now establish the minimality of G . Since G is connected, the removal of a vertex necessarily means the removal of an edge and, therefore, we only need consider the effect of removing or contracting a single edge, e . Because of G 's symmetry, we may assume that either (1) e is in G_1 , or (2) $e = vv_1$.

Suppose (1) holds. Let G'_1 and G' denote the minors of G_1 and G , respectively, that are obtained by the removal or contraction of e (for notational simplicity in the case of a contraction, we insist that e be contracted to v_1 if $v_1 \in e$). Because G_1 is minimal for parameter k , G'_1 possesses a permutation M'_1 with cost at most k . Let M denote the permutation $M_2, vv_2, M'_1, vv_3, M_3$. A column in M_2 or M_3 has cost at most $k + 1$.

Columns vv_2 and vv_3 each have cost two. Any column in M'_1 incurs one additional fill-in (in row v), bringing its cost to at most $k + 1$. Thus, M has cost $k + 1$. We form M' from M at no extra cost by placing vv_1 adjacent to an arbitrary column in M with a 1 in row v_1 .

Suppose (2) holds. If $G' = G \setminus \{e\}$, let M' denote the permutation $M_2, vv_2, vv_3, M_3, M_1$. Since v now has degree two, no fill-ins are required in its row. Because of the way M_2 and M_3 were chosen, any column that requires a new fill-in in row v_2 or row v_3 has cost at most $k + 1$, and so M' has cost $k + 1$. If G' is obtained from G by contracting e to v_1 , let M' denote the permutation $M_2, v_1v_2, M_1, v_1v_3, M_3$. Only the v_i rows may require additional fill-ins. Again, because of the way each M_i was chosen, any new fill-in must lie in a column that has cost at most $k + 1$, and so M' has cost $k + 1$.

In any case, M' is a permutation for G' , and thus the cost of G' is strictly less than that of G . $\square$

Now, if T is a tree obstruction for $k = \text{GML}$, then $\text{gml}(T) = k + 1$, and Corollary 4.2 guarantees there is a vertex $v \in T$ that induces three or more subtrees each of cost k . In light of Theorem 4.3, there must be exactly three such subtrees, each of which is an obstruction for $(k - 1)$ -GML. Therefore, for any $k \geq 2$, all tree obstructions for k -GML can be obtained from the tree obstructions for $(k - 1)$ -GML by using the construction of [4] and Theorem 4.3.

Due to the inductive nature of the construction it is clear that the number of vertices in a tree obstruction must obey the recurrence relation $v(k) = 3v(k - 1) + 1$ with $v(1) = 2$. This recurrence has solution $v(k) = \lfloor 52d3^k/6 \rfloor$ [3], where $v(k)$ is the number of vertices in a k -GML tree obstruction. It is conjectured that the trees are the largest elements in each obstruction set. Since, for each value of k , all tree obstructions have the same order, a proof of the conjecture would mean that every obstruction set is (in principle) computable.

Finally, as a result of Theorem 4.3, we observe that the number of tree obstructions, and hence the size of the obstruction sets for these prob-

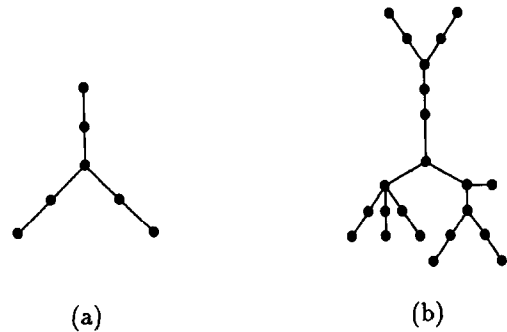


Fig. 1. (a) $S(K_{1,3})$. (b) A 3-GML tree obstruction.

lems, grows rapidly as a function of k . The graph consisting of a single edge is the only obstruction for 1-GML and $S(K_{1,3})$, the graph resulting from subdividing each edge of $K_{1,3}$ with a new degree two vertex, is the only tree obstruction for 2-GML [1,4]. As illustrated in Fig. 1, in constructing tree obstructions for 3-GML, any one of three vertices of $S(K_{1,3})$ may be made adjacent to a new vertex v . This gives ten tree obstructions when $k = 3$. An examination of those obstructions reveals that a total of 88 distinct vertices are found that may be attached to v given $\binom{88}{1} + 2\binom{88}{2} + \binom{88}{3} = 117,480$ tree obstructions for 4-GML.

Acknowledgment

We would like to thank Michael Langston for his valuable suggestions. We would also like to thank the referee for comments and criticism that led to an improvement of the presentation of these results.

References

- [1] R.L. Bryant, M.R. Fellows, N.G. Kinnarsley and M.A. Langston, On finding obstruction sets and polynomial-time algorithms for gate matrix layout, in: *Proc. 25th Allerton Conf. on Communication, Control, and Computing* (1987) 397–398.
- [2] N. Deo, M.S. Krishnamoorthy and M.A. Langston, Exact and approximate solutions for the gate matrix layout problem, *IEEE Trans. Computer-Aided Design* 6 (1987) 79–84.

- [3] J.A. Ellis, I.H. Sudborough and J.S. Turner, Graph separation and search number, Tech. Rept. DCS-66-IR, Dept. of Computer Science, University of Victoria, 1987.
- [4] M.R. Fellows and M.A. Langston, Nonconstructive advances in polynomial-time complexity, *Inform. Process. Lett.* **26** (1987) 157–162.
- [5] M.R. Fellows and M.A. Langston, Nonconstructive tools for proving polynomial-time decidability, *J. ACM* **35** (1988) 727–739.
- [6] M.R. Fellows and M.A. Langston, On search, decision and the efficiency of polynomial-time algorithms, in: *Proc. 21st ACM Symp. on the Theory of Computing* (1989) 501–512.
- [7] N.G. Kinnersley, Obstruction set isolation for layout permutation problems, Ph.D. Thesis, Washington State University, 1989.
- [8] M. Kirousis and C.H. Papadimitriou, Interval graphs and searching, *Discrete Math.* **55** (1985) 181–184.
- [9] M. Kirousis and C.H. Papadimitriou, Searching and pebbling, *Theoret. Comput. Sci.* **47** (1986) 205–218.
- [10] A. LaPaugh, Recontamination does not help to search a graph, Tech. Rept., Electrical Engineering and Computer Science Dept., Princeton University, 1983.
- [11] T. Lengauer, Black-White Pebbles and Graph Separation, *Acta Inform.* **16** (1981) 465–475.
- [12] T.D. Parsons, Pursuit-evasion in a graph, in: Y. Alavi and D.R. Lick, eds., *Theory and Application of Graphs* (Springer, Berlin, 1976) 426–441.
- [13] N. Robertson and P.D. Seymour, Disjoint paths – A survey, *SIAM J. Algebraic Discrete Methods* **6** (1985) 300–305.
- [14] N. Robertson and P.D. Seymour, Graph minors – A survey, in: I. Anderson, ed., *Surveys in Combinatorics* (Cambridge University Press, Cambridge, 1985) 153–171.